

Ponteiros e Funções

ICP141 - Programação de Computadores II

Prof. Marcos Tomazzoli Leipnitz
marcosleipnitz@ic.ufrj.br

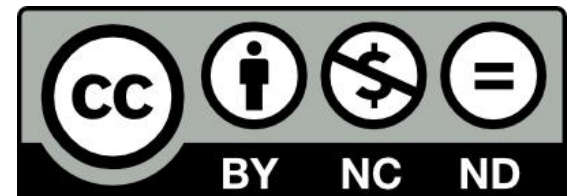


UFRJ

UNIVERSIDADE FEDERAL
DO RIO DE JANEIRO



INSTITUTO DE
COMPUTAÇÃO
UFRJ



- Ponteiros
 - Aritmética de ponteiros
 - Relações e diferenças entre ponteiros e vetores
- Funções
 - Passagem de parâmetros por valor ou referência
- Exemplos
- Exercícios

Adição e Subtração de ponteiros:

- Ponteiro armazena um endereço de memória.
- Podemos **somar** ou **subtrair** valores **inteiros** de ponteiros
- O valor do ponteiro irá **aumentar** ou **diminuir**, dependendo do **número de bytes** que o **tipo base** ocupa, isto é, o **endereço de memória apontado** será deslocado em função do **tamanho do tipo base**.

Regra Geral:

$$[\text{<ponteiro>} \text{ +/- } N] \rightarrow [\text{<endereço inicial>} \text{ +/- } N * \text{<tamanho do tipo>}]$$

Aritmética de Ponteiros

- Considere o programa abaixo:

```
#include <stdio.h>
int main(){

    int vet[3]={10,11,12};
    int *pi; //ptr p inteiro

    //pi aponta para o 1º elemento
    pi=&vet[0];
    printf("pi=%p\n", pi);
    pi++;
    printf("pi=%p\n", pi);
    pi++;
    printf("pi=%p\n", pi);

    return 0;
}
```

Saída:

```
pi=0xffff000bd4
pi=0xffff000bd8
pi=0xffff000bdc
```

Observe que os endereços dos ponteiros incrementaram de 4 bytes, exatamente o tamanho de um inteiro (pode variar dependendo da arquitetura).

<https://pythontutor.com/c.html#mode=edit>

Execução:



Exemplo:

```
#include <stdio.h>
int main(){
    int i, *pi1, *pi2;
    double d, *pd1, *pd2;
    char c, *pc1, *pc2;
    c = 'A';
    d = 5.423;
    i = -20;
    pi1 = &i;
    pi2 = pi1 + 1; // incrementa em uma vez tamanho de um inteiro (4 bytes)
    pd1 = &d;
    pd2 = pd1 + 2; // incrementa em duas vezes o tamanho de um double (2*8 bytes)
    pc1 = &c;
    pc2 = pc1 - 1; // decrementa em uma vez de um char um char (1 byte)
    printf("Resultados em hexadecimal\n");
    printf("pi1: %p, pi2: %p\n", pi1, pi2);
    printf("pd1: %p, pd2: %p\n", pd1, pd2);
    printf("pc1: %p, pc2: %p\n\n", pc1, pc2);

    return 0;
}
```

Resultados em hexadecimal

```
pi1: 0x7ffe07966ae8, pi2: 0x7ffe07966aec
pd1: 0x7ffe07966ad0, pd2: 0x7ffe07966ae0
pc1: 0x7ffe07966abf, pc2: 0x7ffe07966abe
```

Relações entre ponteiros e vetores

- O nome de um vetor corresponde a um **ponteiro para o seu primeiro elemento**.

`int n[50];`  `n` é um ponteiro equivalente a `&n[0]`.

- Diversas operações válidas para ponteiros podem ser executadas com vetores. Assim, as notações abaixo são equivalentes:

Assumindo: `int v[100];`

Com vetores

`v[i]`
`&v[i]`



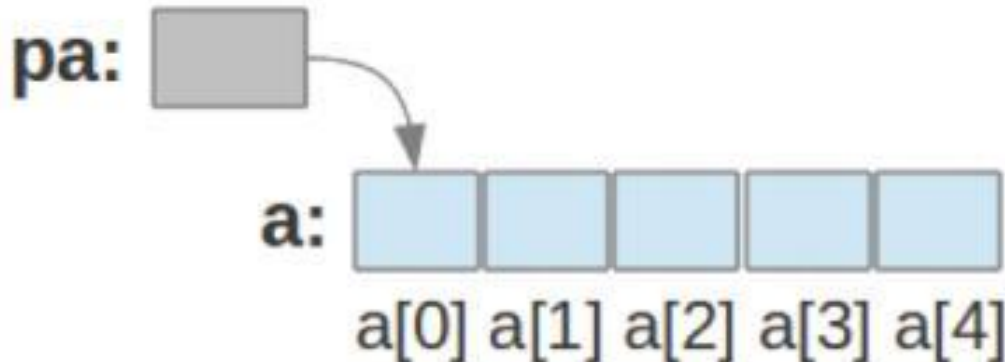
Com ponteiros

`*(v+i)`
`v+i`

Relações entre ponteiros e vetores

Operações que podem ser feitas com vetores também podem ser feitas usando ponteiros:

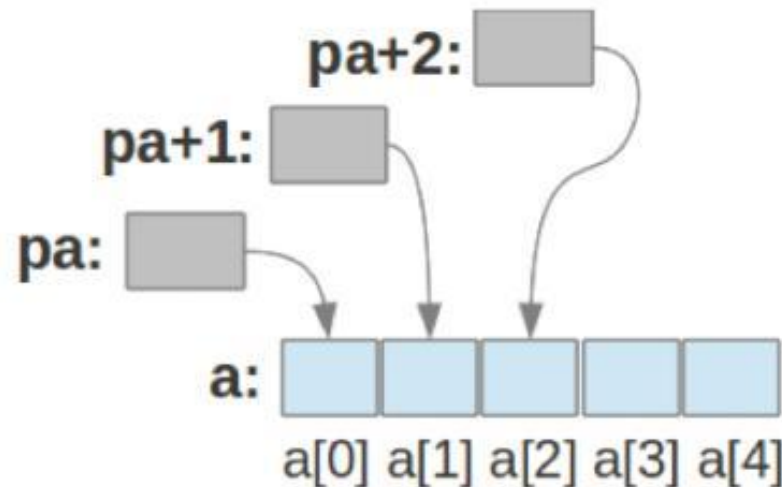
```
int a[5]; //definiu um vetor a de tamanho 5
int *pa;
pa = &a[0]; //fez pa apontar para o primeiro elemento de a
int x;
x = *pa; //copia conteúdo de a[0] para x
```



Relações entre ponteiros e vetores

- Se **pa** aponta para um elemento de um vetor, então **pa+1** aponta para o próximo elemento do vetor
 - **pa+i** aponta para o i-ésimo elemento a partir de **pa**
 - **pa-i** aponta para o i-ésimo elemento antes de **pa**

```
//..continuando..se pa aponta para a[0], então  
x = *(pa+1); //x recebe o valor de a[1]  
x = *(pa+i); //x recebe o valor de a[i]
```



Relações entre ponteiros e vetores

Exemplo:

```
#include <stdio.h>

int main()
{
    int v[5];
    int i;
    printf("Leitura com acesso tipo ponteiro\n");
    for (i = 0; i < 5; i++)
        scanf("%d", v+i); // é um ponteiro, não é necessário usar &

    printf("Impressao com acesso tipo ponteiro\n");
    for (i = 0; i < 5; i++)
        printf("%d ", *(v+i));

    printf("\nImpressao com acesso tipo vetor\n");
    for (i = 0; i < 5; i++)
        printf("%d ", v[i]);

    printf("\n");

    return 0;
}
```

```
Leitura com acesso tipo ponteiro
1
8
10
12
5
Impressao com acesso tipo ponteiro
1 8 10 12 5
Impressao com acesso tipo vetor
1 8 10 12 5
```

Diferenças entre ponteiros e vetores

- **Declaração de vetor**
 - O compilador reserva um **bloco de memória** para que o **vetor inteiro** seja **armazenado**, do **tamanho** especificado na declaração (mas não controla se o uso é limitado à área reservada).
- **Declaração de ponteiro**
 - O compilador aloca um ponteiro para apontar para a memória, **sem reservar espaço de memória**.

Exemplo:

```
int v[3]={1,2,3};  
int *p;  
  
p = v; //operação válida. Atribui a p o end do 1º elem de v  
v = p; //operação inválida (pois v aponta para um endereço fixo de memória)
```

Diferenças entre ponteiros e vetores

- O que **não é equivalente** entre vetores e ponteiros:
 - Um ponteiro é uma variável, então fazer `pa=a` ou `pa++` **é permitido!**
 - O nome de um vetor não é uma variável, então fazer `a=pa` ou `a++` **não é permitido!**
- Exemplos:

```
int a[2], *pa, b[2];
a[0]=1; a[1]=2;
pa = a;
printf("a[0]=%d a[1]=%d\n", *(pa), *(pa+1));
pa++;
printf("a[1]=%d\n", *(pa));
//a++; //error: lvalue required as increment operand
//b = pa; //error: incompatible types when assigning
//...to type 'int[2]' from type 'int *'
```

- **Funções void (sem retorno, ou procedimentos):** não retornam valor associado à função.
 - Sem parâmetros
 - Com parâmetros (**valor ou referência**)
- **Funções tipadas ou com retorno:** retornam um único valor associado a execução da função, usando o comando return.
 - Sem parâmetros
 - Com parâmetros (**valor ou referência**)

Tipos de Funções e Parâmetros

- Procedimentos e funções podem receber parâmetros, que são passados pela função ou procedimento que realiza a chamada.
- Parâmetros são o meio através do qual diferentes funções compartilham dados entre si.
- Parâmetros permitem o desenvolvimento de funções mais genéricas, que produzem diferentes resultados/comportamentos, dependendo dos dados recebidos como parâmetros.

Passagem de parâmetros por valor

- Apenas um **valor** é passado para a função.
- Valores dos **parâmetros reais** ou argumentos utilizados na chamada são **copiados** para os **parâmetros formais** da função, que podem ser usados como **variáveis locais** da função.
- Alteração no valor dos parâmetros dentro do procedimento ou função altera apenas as variáveis locais, ou seja, **não altera o valor** das variáveis passadas na chamada da função.

Passagem de parâmetros por valor

- Exemplo: Fazer uma função que receba dois inteiros **x** e **y** e troque seus valores.

```
#include <stdio.h>
void troca(int x, int y)
{
    int temp;

    temp = x;
    x = y;
    y = temp;
}
```

```
int main()
{
    int x=5, y=10;

    printf("x=%d y=%d\n",x,y);
    troca(x,y);
    printf("x=%d y=%d\n",x,y);
    return 0;
}
```

O que será impresso na tela?

Passagem de parâmetros por valor

```
x=5 y=10  
x=5 y=10
```

- Os valores de **x** e **y** não foram trocados porque a passagem de parâmetros foi feita por valor.
- Passagem de parâmetros por valor:
 - Variável local **x** é inicializada com conteúdo de **x** (argumento);
 - Variável local **y** é inicializada com conteúdo de **y** (argumento);
 - Dentro da função, os valores de **x** e **y** são trocados, mas esta mudança é feita sobre as **cópias locais dos valores** e não é propagada para a **main**, onde **x** e **y** permanecem com os valores originais.

Passagem de parâmetros por referência

- Não passa uma cópia do valor.
- Passa um endereço (referência) de memória, o qual é associado ao parâmetro, localmente.
- Alterações feitas no parâmetro afetam o conteúdo do endereço referido na chamada (argumento ou parâmetro real).
- Permite que funções/procedimentos façam alterações nos parâmetros, que são propagadas para a função/procedimento que realizou a chamada.
- Para passar parâmetros por referência, usamos ponteiros!

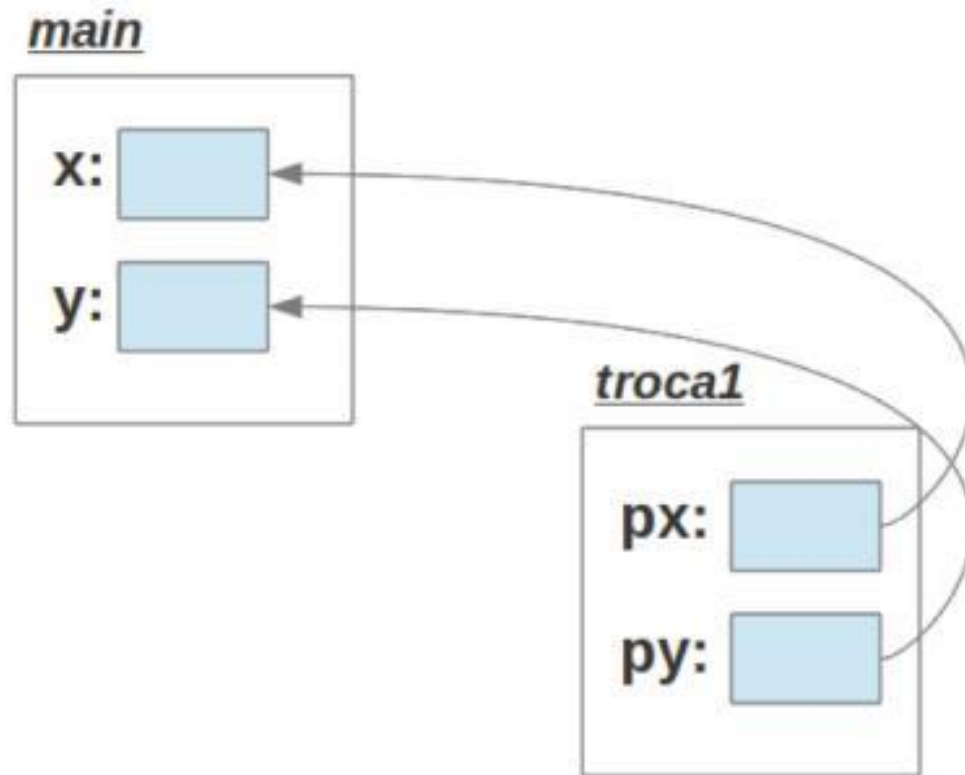
- Qual é a grande **vantagem** de usarmos ponteiros para realizar a passagem de parâmetros?
 - Ao invés de cópias dos valores, são passadas **referências** (ponteiros) para estes valores.
 - Assim, **os valores podem ser modificados** dentro da função/procedimento.
 - Os efeitos dessa modificação são **propagados** para as variáveis da função que realizou a chamada da função.

```
//exemplo de passagem de argumento por valor!  
void troca (int x, int y);
```

```
//exemplo de passagem de argumento por referência!  
void troca1 (int *px, int *py);
```

Passagem de parâmetros por referência

- Argumentos do tipo **ponteiro** permitem que a função acesse (e **modifique**) diretamente os **objetos/variáveis** da função que a chamou



Passagem de parâmetros por referência

- Exemplo: Fazer uma função que receba dois inteiros **x** e **y** e troque seus valores.

```
#include <stdio.h>
void troca1(int *px, int *py)
{ // conteúdo dos endereços
    int temp;

    temp=*px;
    *px = *py;
    *py = temp;
}
```

```
int main()
{
    int x=5, y=10;

    printf("x=%d y=%d\n",x,y);
    troca1(&x,&y); //aqui,
    endereços
    printf("x=%d y=%d\n",x,y);
    return 0;
}
```

Passagem de parâmetros:

- Agora **px** e **py** recebem os endereços de **x** e **y**
- Na função, valores são buscados e armazenados no endereço referido, isto é, em **x** e **y**!

Passagem de parâmetros por referência

```
x=5 y=10  
x=10 y=5
```

- Os valores **foram trocados**, pois a passagem de parâmetros foi feita por **referência**.
- Dentro da função, **px** e **py** receberam os **endereços de memória** de **x** e **y**, que têm os seus **conteúdos** alterados.

Passagem de parâmetros por referência

- Outro exemplo: desenvolver um procedimento que **incrementa** o valor recebido como parâmetro.

```
#include<stdio.h>
void incrementa(int *x){
    *x = *x+1;
}

int main(){
    int n = 0;
    printf("Antes: %d\n",n);
    incrementa(&n);
    printf("Depois: %d\n",n);
    return 0;
}
```

Indica que a função recebe um **ponteiro para int**.

Indica que estamos passando um **ponteiro** para a variável **n** como **parâmetro real** para a função incrementa.